UNIVERSIDAD DE GUADALAJARA
Centro Universitario de Ciencias Exactas e Ingenierías



Act. 5: Técnica Voraz Huffman

Por:
OCTAVIO EMMANUEL LOPEZ ORTIZ
ANDREA VALERIA TORRES FIGUEROA
ESTEFANIA NAVARRO MENDOZA

Docente:
JORGE ERNESTO LOPEZ ARCE DELGADO

1.1 Introducción

El presente proyecto consiste en el desarrollo de una plataforma web interactiva y modular diseñada para servir como herramienta de análisis y consulta didáctica. La aplicación integra visualizaciones gráficas para algoritmos de ordenamiento, estructuras de datos avanzadas, técnicas de optimización voraz, sucesiones matemáticas y un sistema automatizado de descarga y compresión de textos literarios basados en el estándar de Huffman. Mediante el uso de tecnologías web modernas, se logra un entorno accesible directamente desde el navegador, eliminando la necesidad de instalaciones complejas y facilitando la experimentación directa con los tiempos de ejecución y las estructuras de información.

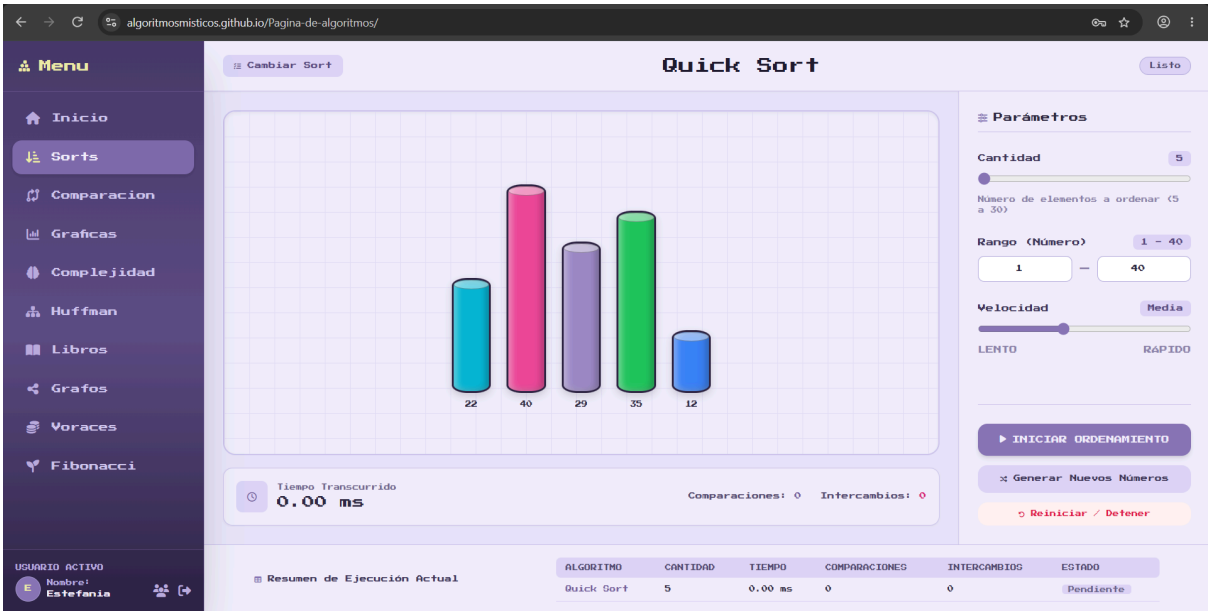


Figura 1: *Vista general del módulo de algoritmos de ordenamiento (Quick Sort), donde se aprecian la representación gráfica de barras dinámicas, los controles de simulación y el resumen de ejecución actual.*

1.2 Objetivo General

Construir una herramienta web interactiva basada en tecnologías del lado del cliente (HTML5, Tailwind CSS y JavaScript) que permita simular, visualizar y procesar algoritmos de ordenamiento, estructuras de grafos, técnicas voraces, cálculos de Fibonacci y compresión de archivos de texto mediante el algoritmo de Huffman con una extensión propia.

1.3 Objetivos Específicos

- Diseñar una interfaz de usuario intuitiva, estética y adaptable (*responsive*) que integre paneles de control parametrizables para la manipulación de datos y velocidades de simulación.
- Implementar la representación gráfica en tiempo real de algoritmos de ordenamiento de complejidad cuadrática ($O(n^2)$) y logarítmica ($O(n \log n)$), destacando comparaciones e intercambios.
- Incorporar un módulo de conectividad con repositorios externos (Project Gutenberg y Gutendex) respaldado por un proxy intermediario (gutenberg-proxy-worker.js) para solventar restricciones de seguridad CORS durante la descarga de libros en formato de texto plano.

- Desarrollar la lógica de compresión mediante el algoritmo de Huffman, permitiendo la construcción de tablas de frecuencias, la generación de códigos binarios optimizados y la exportación bajo la extensión propietaria .algomistico.
- Integrar el análisis de estructuras fundamentales como grafos, algoritmos basados en técnicas voraces y secuencias de Fibonacci dentro de la arquitectura del proyecto.

2.1 Fundamentos y Algoritmos Implementados

El módulo de ordenamiento de la plataforma permite comparar el comportamiento y la eficiencia de diferentes algoritmos clásicos mediante representaciones visuales dinámicas. Entre las estrategias implementadas se encuentran métodos basados en comparaciones sucesivas y particionamiento recursivo:

- **Bubble Sort (Ordenamiento de Burbuja):** Algoritmo iterativo que compara elementos adyacentes y los intercambia si se encuentran en el orden incorrecto, repitiendo el proceso hasta que la lista queda completamente ordenada. Presenta una complejidad temporal de $O(n^2)$ en el peor de los casos.
- **Quick Sort (Ordenamiento Rápido):** Algoritmo de divide y vencerás que selecciona un elemento pivote, particiona el arreglo reorganizando los elementos menores a la izquierda y los mayores a la derecha, y aplica el proceso de forma recursiva. Su complejidad promedio es de $O(n \log n)$.
- **Merge Sort (Ordenamiento por Mezcla):** Estrategia de divide y vencerás que divide el arreglo original en mitades de forma recursiva hasta obtener subarreglos de un solo elemento, para posteriormente fusionarlos de manera ordenada. Su complejidad temporal se mantiene estricta en $O(n \log n)$ tanto en el peor como en el mejor de los casos.

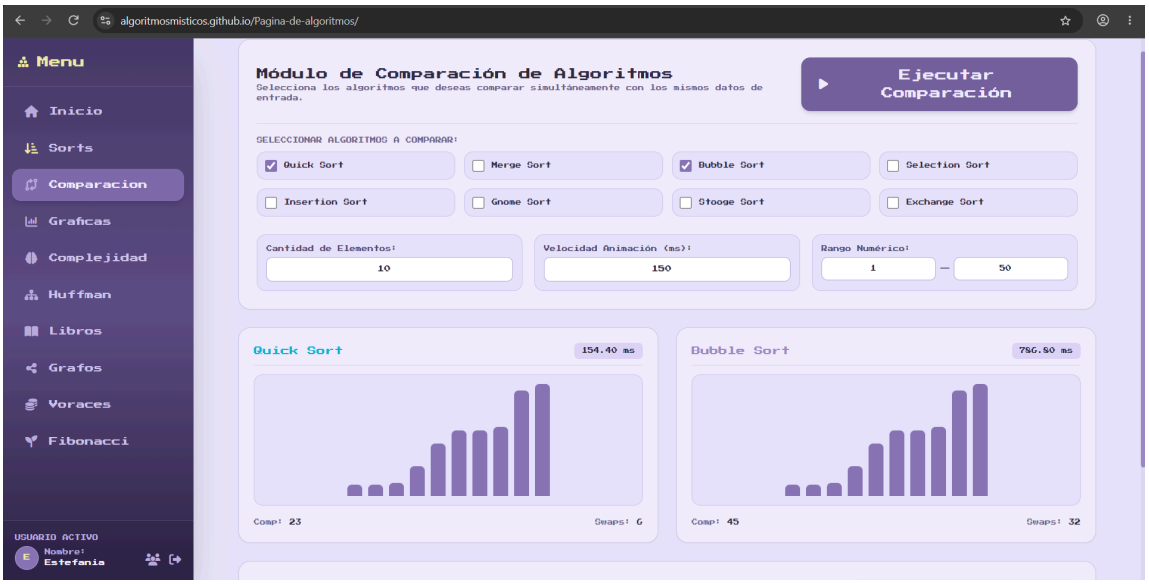


Figura 2: *Vista general de la comparativa de métodos de ordenamiento en la plataforma web, donde se contrasta el rendimiento visual y cuantitativo de algoritmos con diferente complejidad temporal.*

2.2 Panel de Control y Parámetros de Simulación

La interfaz de usuario incluye controles interactivos que permiten al estudiante o investigador modificar en tiempo real los parámetros del experimento:

- **Cantidad de elementos:** Permite definir el volumen de datos a procesar en un rango preestablecido.
- **Rango numérico:** Establece los límites inferior y superior de los valores aleatorios generados para las barras.
- **Velocidad de ejecución:** Ajusta el retardo temporal entre las comparaciones e intercambios para visualizar paso a paso el comportamiento del algoritmo.
- **Métricas en tiempo real:** Muestra de manera concurrente el tiempo transcurrido (en milisegundos), el número total de comparaciones realizadas y los intercambios efectuados.

3.1 Fundamentos del Algoritmo de Huffman

El módulo de compresión de la plataforma implementa el **algoritmo de Huffman**, un método de codificación voraz diseñado por David Huffman para la compresión sin pérdida de datos. La premisa fundamental del algoritmo consiste en asignar códigos de longitud variable a los diferentes símbolos o caracteres de un texto, de modo que aquellos caracteres con mayor frecuencia de aparición reciban códigos binarios más cortos, mientras que los caracteres menos frecuentes utilicen secuencias más largas.

El proceso algorítmico se estructura de la siguiente manera:

1. **Análisis de frecuencias:** Se procesa la cadena de texto de entrada para contabilizar la repetición exacta de cada carácter único.
2. **Empaquetado y Extensión Propia (.algomistico):** Una vez codificado el texto, el sistema permite exportar el resultado utilizando una extensión propietaria diseñada para este proyecto, denominada **.algomistico**, la cual encapsula tanto la estructura de decodificación como la secuencia binaria comprimida.
3. **Construcción del árbol binario:** Se crea un árbol de nodos ponderados donde cada hoja representa un carácter y su respectiva frecuencia. De manera iterativa, se combinan los dos nodos con menor frecuencia hasta formar un único árbol raíz.
4. **Asignación de códigos:** Se recorren las ramas del árbol (asignando típicamente '0' a los enlaces izquierdos y '1' a los derechos) para determinar el código binario definitivo de cada símbolo.

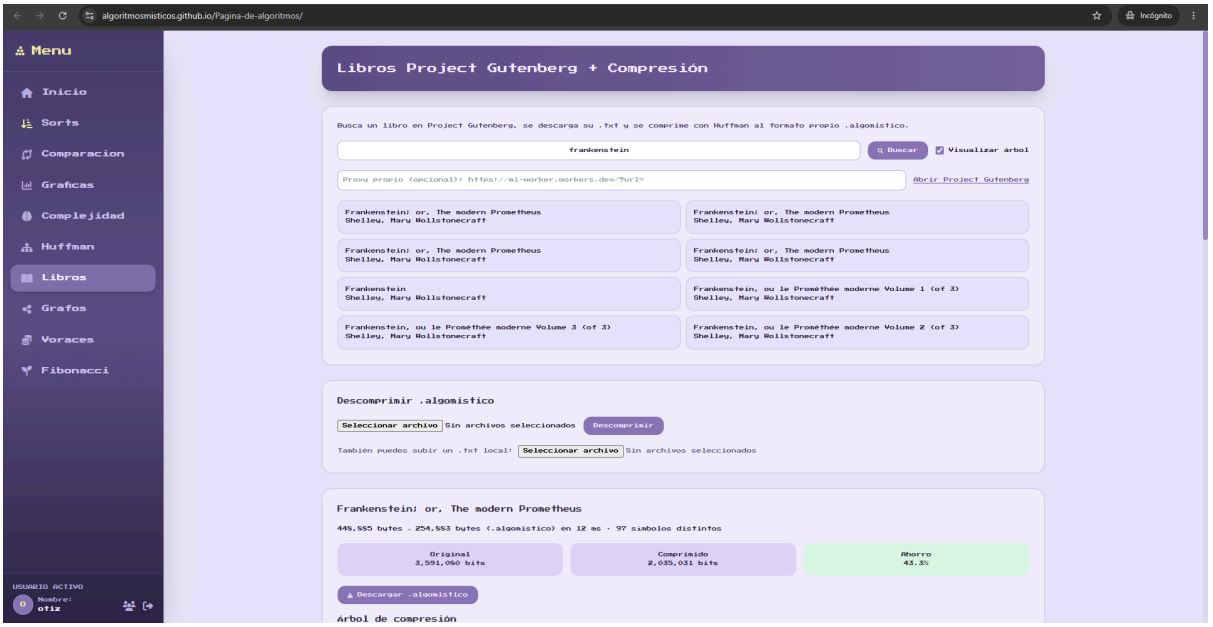


Figura 3.1: *Vista general de la interfaz de búsqueda de libros utilizando el algoritmo de Huffman, mostrando el apartado de búsqueda y su empaquetado y extensión propia con el formato .algomistico.*

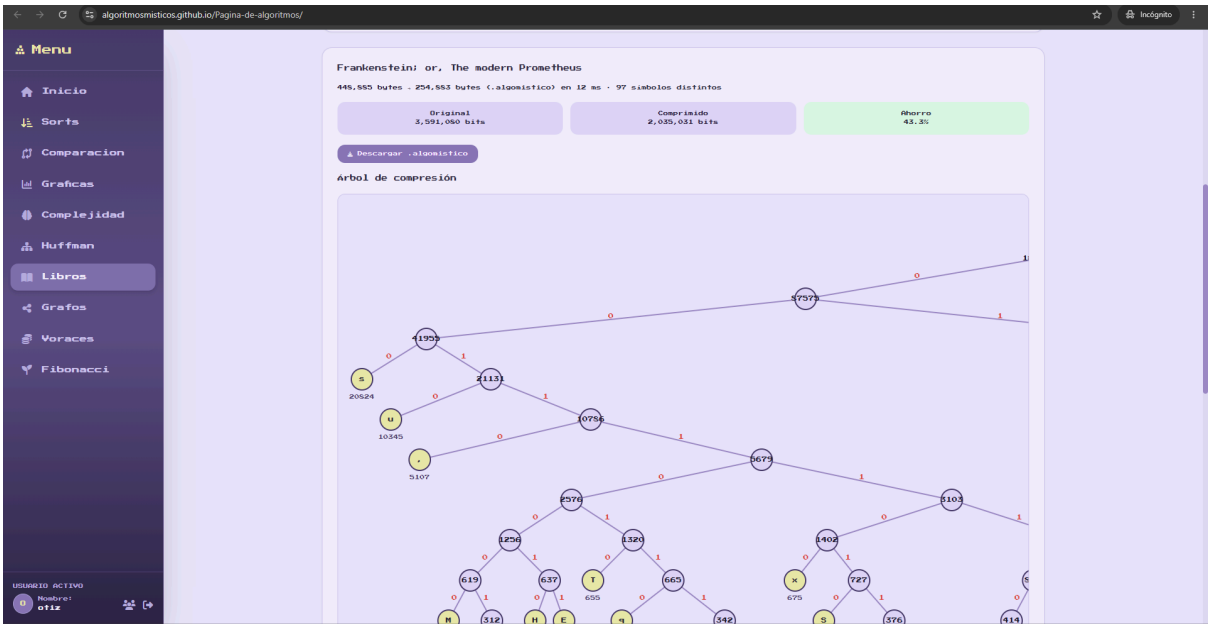


Figura 3.2: *Vista general del lado izquierdo del árbol binario generado a través del libro seleccionado anteriormente.*

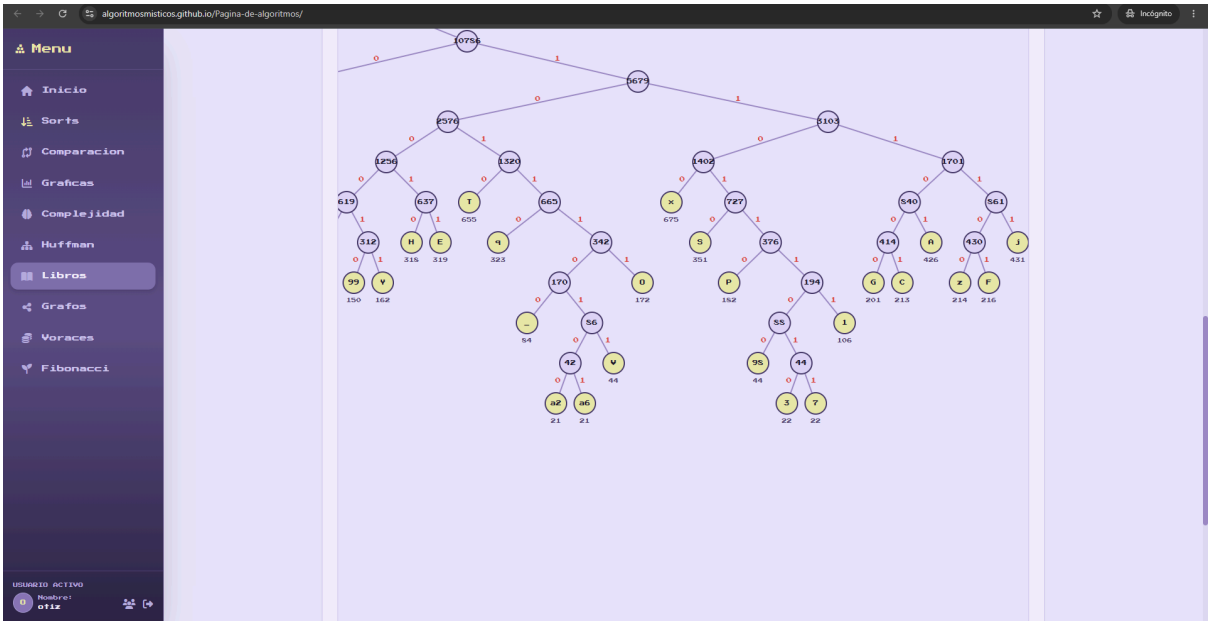


Figura 3.3: *Vista general del lado derecho del árbol binario generado a través del libro seleccionado anteriormente.*

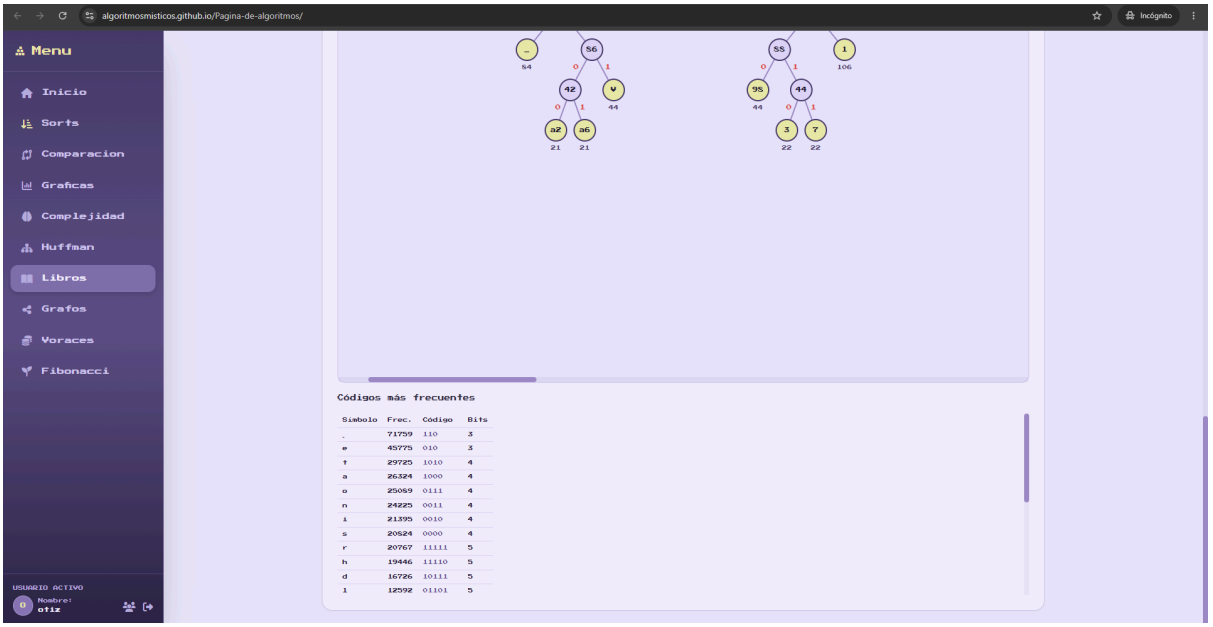


Figura 3.4: *Vista general de la tabla binaria utilizando las ramas del árbol para determinar el código binario definitivo de cada símbolo.*

3.2 Integración con Project Gutenberg y el Servidor Proxy

Para dotar al sistema de textos literarios extensos y reales sin depender exclusivamente de muestras estáticas, la plataforma incorpora un módulo de conectividad directa con **Project Gutenberg** y su motor de búsqueda de metadatos **Gutendex**.

Dado que las peticiones directas desde el navegador hacia repositorios externos suelen encontrarse bloqueadas por políticas de seguridad de origen cruzado

(**CORS**), la arquitectura del proyecto integra un script intermediario (**gutenberg-proxy-worker.js**). Este componente opera como un servidor proxy que gestiona la descarga segura de los libros en formato de texto plano (.txt), permitiendo que el usuario seleccione una obra clásica, la importe automáticamente a la interfaz y aplique de inmediato la tabla de frecuencias, la visualización del árbol de Huffman y la compresión en formato .algomistico.

4.1 Módulo de Grafos y Recorridos

Los grafos representan estructuras de datos no lineales compuestas por vértices (o nodos) y aristas (relaciones o conexiones entre ellos). En el contexto de la plataforma, este módulo permite modelar redes de interconexión y aplicar algoritmos fundamentales de exploración y optimización:

- **Representación visual y matricial:** La interfaz permite visualizar la disposición de los nodos, facilitando el análisis de conectividad y adyacencia.
- **Algoritmos de búsqueda:** Se integran estrategias de recorrido como la **Búsqueda en Anchura (BFS - *Breadth-First Search*)** y la **Búsqueda en Profundidad (DFS - *Depth-First Search*)**, las cuales permiten examinar sistemáticamente las rutas y componentes conexas del grafo.

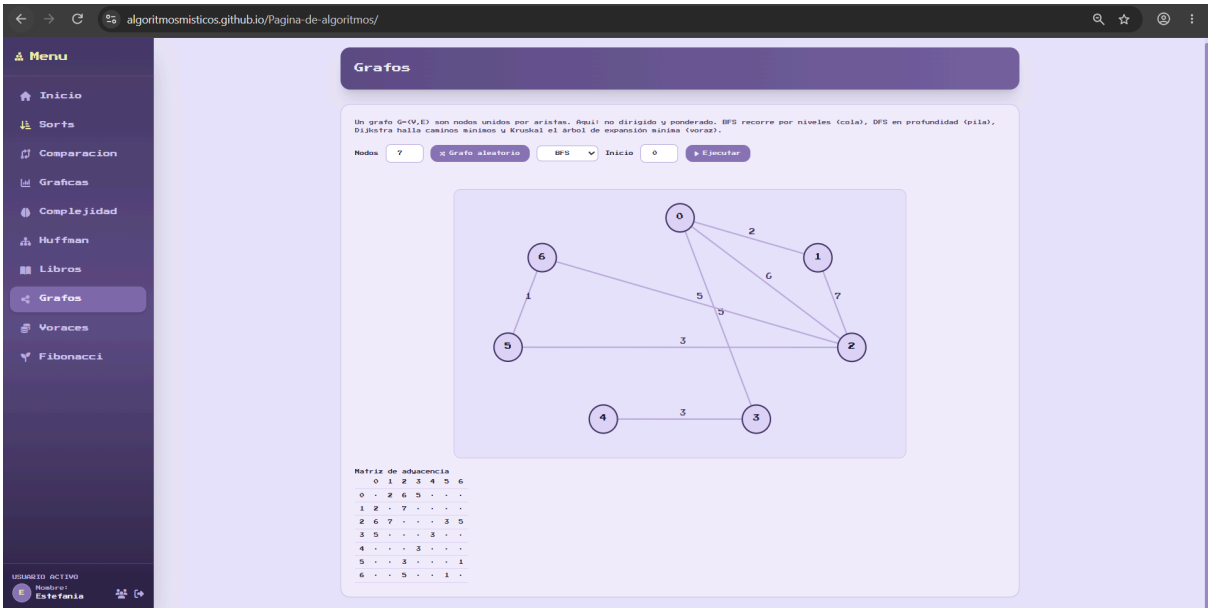


Figura 4: *Módulo de representación y recorrido de grafos, mostrando la red de nodos interconectados con pesos en sus aristas, junto con la matriz de adyacencia generada en la parte inferior.*

4.2 Técnicas Voraces (Algoritmos *Greedy*)

Las técnicas voraces constituyen un paradigma de diseño algorítmico que toma decisiones óptimas en cada etapa local con la esperanza de alcanzar una solución óptima global. Dentro de la plataforma, estos conceptos se aplican en la resolución de problemas de optimización:

- **Fundamento teórico:** A diferencia de la programación dinámica o la fuerza bruta, un algoritmo voraz no reconsidera sus elecciones pasadas.
- **Aplicaciones prácticas:** El ejemplo más claro implementado en la arquitectura del proyecto es el propio **algoritmo de Huffman**, el cual selecciona de manera voraz los dos nodos con menor frecuencia para construir el árbol óptimo de codificación. Asimismo, se estudian enfoques clásicos de selección de recursos y optimización de rutas.

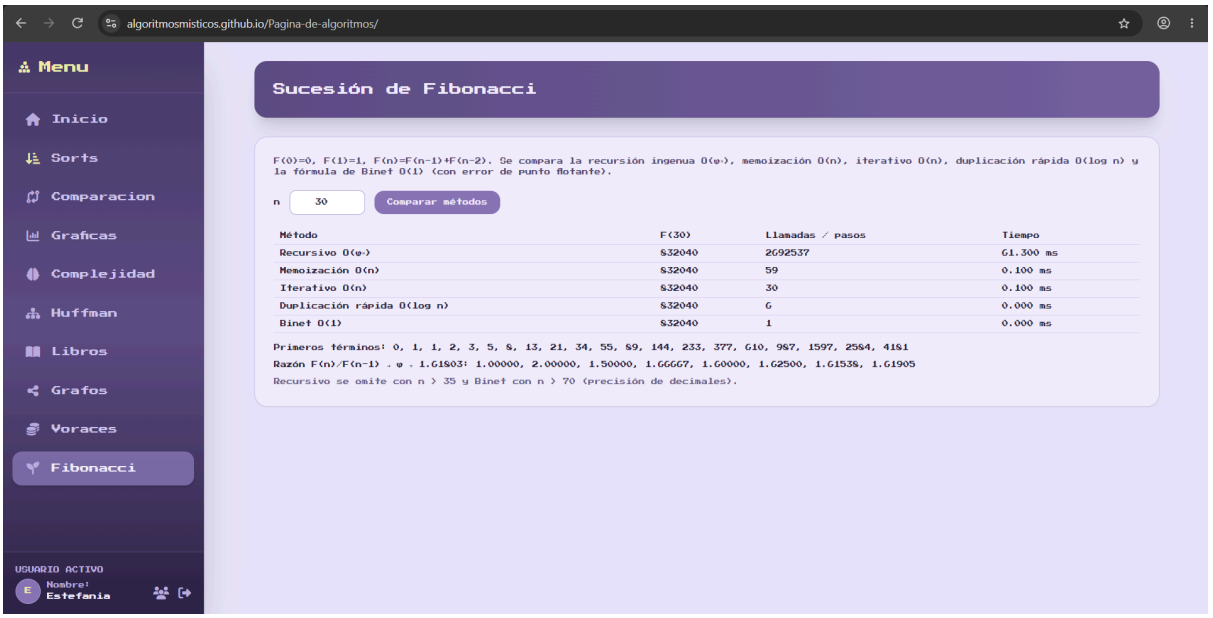


Figura 5: *Comparativa de eficiencia computacional en el cálculo de la sucesión de Fibonacci ($n = 30$), evaluando métricas de tiempo, llamadas y pasos entre enfoques recursivos, memoización, iterativos, duplicación rápida y la fórmula de Binet.*

4.3 Sucesión de Fibonacci y Eficiencia Computacional

El módulo dedicado a la **sucesión de Fibonacci** permite analizar el impacto del diseño algorítmico:

- **Definición matemática:** La serie se define de manera recursiva donde cada término es la suma de los dos anteriores ($F_n = F_{n-1} + F_{n-2}$).
- **Enfoque de resolución:** La plataforma contrasta la implementación recursiva simple (con una complejidad exponencial $O(2^n)$ y problemas de redundancia) frente a enfoques optimizados mediante iteración o memorización (con complejidad lineal $O(n)$).

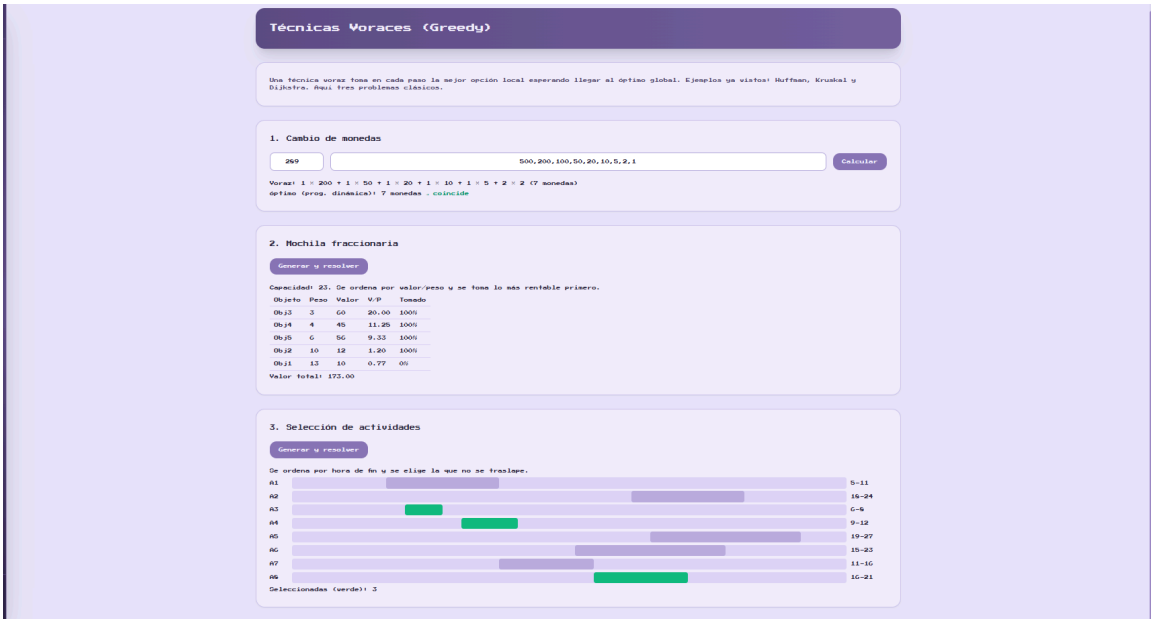


Figura 6: *Módulo de algoritmos voraces (Greedy), ilustrando la resolución interactiva de problemas clásicos de optimización: cambio de monedas, el problema de la mochila fraccionaria (Fractional Knapsack) y la selección de actividades.*

5.1 Conclusiones

El desarrollo e implementación de la plataforma web interactiva de algoritmos ha permitido consolidar de manera práctica los conceptos teóricos fundamentales de la ciencia de la computación. A lo largo del proyecto, se cumplieron los objetivos planteados al lograr:

- **Visualización Dinámica:** La representación gráfica en tiempo real de los algoritmos de ordenamiento (*Quick Sort*, *Bubble Sort*, *Merge Sort*) facilitó la comprensión intuitiva de su complejidad temporal y espacial, superando las limitaciones de la ejecución estática en consola.
- **Integración y Procesamiento de Datos Externos:** Mediante la conectividad respaldada con Project Gutenberg y Gutendex a través del servidor proxy, se logró simular escenarios reales de procesamiento de textos masivos aplicando el algoritmo de compresión voraz de Huffman y su exportación mediante la extensión propietaria `.algomistico`.
- **Análisis Estructural y de Optimización:** La inclusión de módulos para grafos, técnicas voraces y el cálculo comparativo de la sucesión de Fibonacci demostró cómo la elección del paradigma de programación impacta de forma directa en el rendimiento y la eficiencia de los recursos computacionales.

En consecuencia, el sistema se establece como una solución robusta, accesible y de alta utilidad académica para el estudio y experimentación de estructuras de datos y algoritmos avanzados.

5.2 Referencias Bibliográficas

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3.ª ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.ª ed.). Addison-Wesley Professional.
- Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3.ª ed.). Addison-Wesley.
- Project Gutenberg. (2026). *Free eBooks - Project Gutenberg*. Recuperado de https://www.gutenberg.org/